Layer Normalization and Overestimation Bias in DDPG and TD3 on LunarLander

Anna Krasovskaya Arghavan Aslani

1 Introduction

Off-policy actor-critic algorithms such as DDPG [4] learn continuous-control policies by training a critic $Q_\phi(s, a)$ with bootstrapped temporal-difference targets, and an actor $\mu_\theta(s)$ that maximizes this critic. Because the actor is optimized against an approximate critic, it is drawn towards actions whose value is overestimated, and bootstrapping propagates these errors to other states: the critic's estimates become systematically larger than the true returns, a phenomenon known as overestimation bias [3]. TD3 [3] reduces this bias with algorithmic changes, most notably by computing the critic targets from the minimum of two critics (clipped double Q-learning).

A different line of work modifies the critic's architecture instead. Layer normalization [1] rescales the activations of each hidden layer, which keeps the critic's internal features at a fixed scale and may limit how far its estimates extrapolate on actions that are rarely seen in the data. Ball et al. [2] report that layer normalization mitigates catastrophic overestimation when learning with prior data, and Nauman et al. [5] find it more effective at reducing overestimation than techniques designed for that purpose, such as clipped double Q-learning. These results were obtained with SAC, on robotic control benchmarks and often with many gradient updates per environment step; whether they transfer to DDPG and TD3 in a standard setting is less clear.

In this study, we train DDPG and TD3, each with and without layer normalization in the critic, on the LunarLanderContinuous environment, and measure overestimation by comparing the critic's estimates with Monte Carlo returns from the same state-action pairs. We address two questions: (Q1) does layer normalization reduce the overestimation bias of DDPG and TD3? (Q2) is a larger overestimation bias associated with lower performance?

Based on these studies, we expect layer normalization to reduce the overestimation bias of DDPG (H1a). For TD3, whose clipped double Q-learning already counteracts overestimation, we expect a smaller effect, since less bias remains to be removed (H1b). Following Fujimoto et al. [3], who show that overestimation leads the actor towards actions whose value is overestimated, we expect runs with a larger overestimation bias to reach a lower performance (H2).

Our code, results and analysis scripts are available at `https://github.com/AnnaENS/lunar-lander-RLproject`

2 Methods

2.1 Environment

We use the LunarLanderContinuous-v3 environment from Gymnasium 1.3.0 [6], in which an agent must land a spacecraft on a landing pad. Each episode starts with the lander at the top of the screen and a random initial force applied to it. The state is an 8-dimensional vector containing the lander's horizontal and vertical position and velocity, its angle and angular velocity, and two binary indicators of leg contact with the ground. The action is a 2-dimensional continuous vector in $[-1, 1]^2$ that sets the throttle of the main engine and of the lateral engines. At each step, the reward increases as the lander moves closer to the pad, slows down and stays level, gives $+10$ for each leg touching the ground, and penalizes engine use in proportion to the throttle (up to 0.3 per step for the main engine and 0.03 for the lateral engines). Episodes terminate when the lander crashes or leaves the screen, with a final reward of -100, or comes to rest, with a final reward of $+100$; they are truncated after 1000 steps. An episode return of at least 200 is considered a solution. Critic targets are bootstrapped after a truncation but not after a termination.

2.2 Algorithms

Both algorithms are implemented in PyTorch with the rl-mind toolkit (v0.3.0), following the DDPG/TD3 notebook of the course practicals. **DDPG** [4] learns a deterministic actor $\mu_\theta(s)$ and a critic $Q_\phi(s, a)$. The critic minimizes the squared temporal-difference error with respect to the target

$$y = r + \gamma(1 - d)\, Q_{\phi'}\big(s', \mu_\theta(s')\big), \tag{1}$$

Table 1: Hyperparameters, identical across the four conditions.

Hyperparameter	Value	Hyperparameter	Value
Training steps per run	300,000	Hidden layers (actor and critics)	2×256, ReLU
Steps before the first update	10,000	Optimizer (actor and critics)	Adam, learning rate $3 \cdot 10^{-4}$
Gradient updates per env. step	1	Exploration noise	$\mathcal{N}(0, 0.1)$
Replay buffer size	10^6	TD3 target policy noise, clip	$\mathcal{N}(0, 0.2)$, ± 0.5
Mini-batch size	256	TD3 policy delay	2
Discount factor γ	0.99	Evaluation	every 10^4 steps, 10 episodes, no noise
Target update rate τ	0.005	Bias measurement	after each evaluation, 5 episodes, no noise
Seeds	1–10 per condition	Truncation margin (MC returns)	500 steps

where d indicates that s' is terminal and $Q_{\phi'}$ is a target critic that slowly tracks the critic through soft updates, $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$. The actor is updated to maximize $Q_\phi(s, \mu_\theta(s))$. **TD3** [3] adds three mechanisms: two critics trained towards a common target computed with the minimum of the two target critics (clipped double Q-learning); clipped Gaussian noise added to the target actor's action in this target (target policy smoothing); and actor and target-network updates performed only every two critic updates (delayed policy updates). The actor maximizes the first critic.

The actor and critics are multilayer perceptrons with two hidden layers of 256 ReLU units; the actor ends with a tanh, and the critic takes the concatenated state and action as input. In the **layer normalization** condition, a layer normalization [1] is inserted after each hidden linear layer of the critic(s), before the ReLU; the actor is unchanged. During training, Gaussian noise is added to the actor's actions for exploration, transitions are stored in a replay buffer, and one gradient update on a uniformly sampled mini-batch is performed per environment step, starting after the first 10^4 steps.

2.3 Hyperparameters

All hyperparameters are listed in Table 1 and are identical across the four conditions. We follow Fujimoto et al. [3] for γ, τ, the exploration noise and the TD3-specific parameters, and use values common in later implementations for the network size, learning rate and batch size (the original paper used 400–300 units, a learning rate of 10^{-3} and a batch size of 100). The other hyperparameters suggested in the project (network depth, update-to-data ratio and critic-to-actor update ratio) are kept fixed.

2.4 Measuring overestimation

As in Lillicrap et al. [4] and Fujimoto et al. [3], we estimate the critic's bias by comparing its predictions with observed discounted returns. After each evaluation, i.e. every 10^4 steps, we play 5 episodes with the deterministic actor (no exploration noise), from initial states seeded by the run seed and the training step. For each visited pair (s_t, a_t), we compute the Monte Carlo return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$ backward from the end of the episode, with the same γ as in training. Because critic targets are bootstrapped at truncations, returns of truncated episodes are incomplete near the time limit: we keep all the steps of terminated episodes, but only the steps at least 500 steps before the end of truncated episodes, for which the missing part of the return is weighted by at most $\gamma^{500} \approx 0.007$. The bias is the mean of $Q_\phi(s_t, a_t) - G_t$ over the kept pairs (between 329 and 3929 pairs per measurement, median 1949); positive values indicate overestimation. For TD3, we report the bias of the first critic, which is the one used to update the actor. Using the minimum of the two critics instead changes the final TD3 bias by less than one unit. Unlike Fujimoto et al. [3], who average 1000 rollouts from states sampled from the replay buffer, we use a single rollout per state along the actor's own trajectories, which is cheaper but gives a noisier estimate of the true value.

2.5 Experimental protocol and statistics

We use a 2×2 design (algorithm $\times$ layer normalization) with 10 seeds per condition, as in Fujimoto et al. [3] (seeds 1–10, shared across conditions), i.e. 40 runs of $3 \cdot 10^5$ steps. The seed determines the network initialization, the exploration noise and the environment randomness. Every 10^4 steps, the deterministic actor is evaluated on 10 episodes in separate environment instances, and performance is the mean undiscounted return. For each run, the final performance and the final bias are the means over the last three evaluations (280k to 300k steps); we also report the bias averaged over the evaluations after the first update (20k to 300k steps). Runs are the unit of analysis: learning curves show the mean across seeds with a 95% Student-t confidence interval, and the conditions with and without layer normalization are compared within each algorithm with Welch's t-test and a two-sided Mann–Whitney U test on per-run values. We compare TD3 with DDPG within each layer normalization condition with the same tests. Since this makes 12 comparisons (2 families $\times$ 3 measures $\times$ 2), we consider a difference significant when p is below the Bonferroni-corrected threshold $0.05/12 \approx 0.004$. To relate bias and performance (Q2), we compute the Spearman rank correlation between the final bias and the final performance of the runs, over all 40 runs and within each algorithm (20 runs each), since differences between the two algorithms alone could

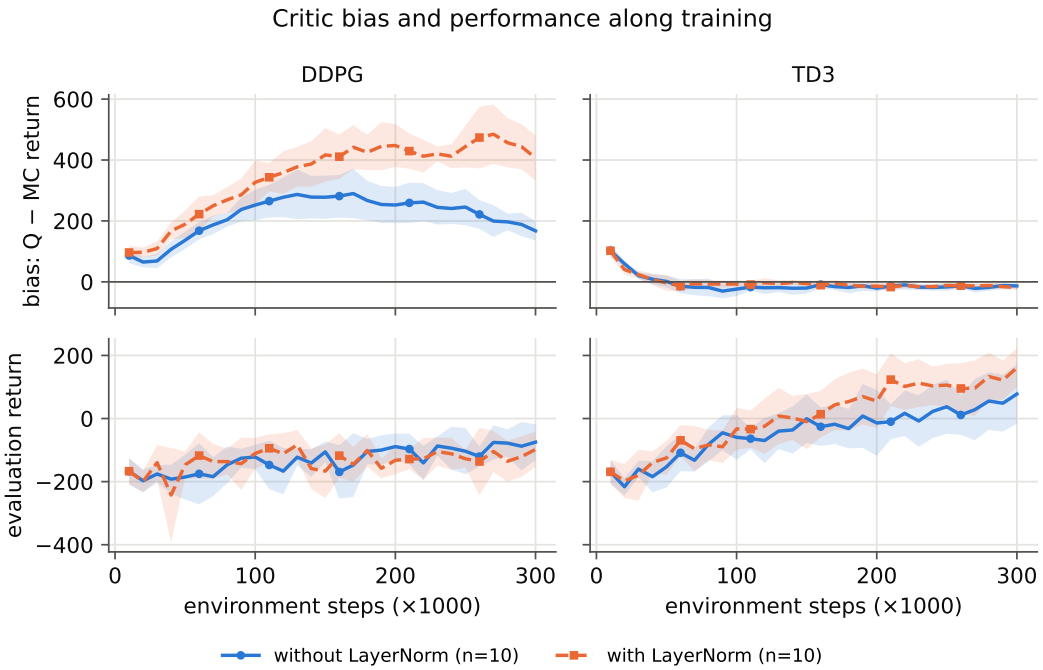


Figure 1: Critic bias (top) and evaluation return (bottom, 10 episodes without noise) along training, for DDPG (left) and TD3 (right), without (solid) and with (dashed) layer normalization in the critic. Mean over 10 seeds, shaded area: 95% confidence interval.

produce a correlation across all runs. We used Python 3.12, PyTorch 2.14.1, Gymnasium 1.3.0 and rl-mind 0.3.0, on a CPU (Apple M3), where a run took 16 to 37 minutes.

3 Results

Table 2 gives the final return and the critic bias of each condition (mean ± std over 10 seeds). We report Welch p-values and consider a difference to be significant when $p < 0.004$ (Bonferroni over the 12 comparisons).

DDPG vs TD3. DDPG overestimates much more than TD3 (Figure 1, top). Both start with a bias of about 90 at 10k steps, when the critic has not been trained yet. The bias of DDPG then increases during the first 100k-150k steps and stays high until the end. The bias of TD3 goes down to about zero in 50k steps and then stays slightly negative, between −30 and 0. The difference is significant with and without layer normalization, for the final bias and for the mean bias (all $p < 10^{-6}$). All DDPG runs end with a higher bias than all TD3 runs.

Layer normalization (Q1). With DDPG, layer normalization increases the bias instead of reducing it. The two curves are close until about 100k steps. After that, the bias without layer normalization peaks at about 290 around 170k steps and goes back down to about 170, while with layer normalization it keeps increasing, reaches about 480 at 270k steps and ends around 410. The final bias goes from 184.3 to 435.7 ($p = 1.4 \cdot 10^{-5}$, Mann-Whitney $p = 3.3 \cdot 10^{-4}$) and the mean bias from 220.1 to 357.1 ($p = 3.2 \cdot 10^{-5}$). Nine of the ten DDPG runs with layer normalization end with a higher bias than every run without it. With TD3, the curves overlap and we see no effect (final bias: $p = 0.87$; mean bias: $p = 0.31$).

Performance. DDPG barely improves: its return stays between −200 and −70 during training (Figure 1, bottom). TD3 improves steadily, from about −200 to about 80 without layer normalization and to about 160 with it. TD3 ends better than DDPG with layer normalization (138.6 vs −118.1, $p = 7 \cdot 10^{-7}$). Without layer normalization the difference is also large (60.6 vs −80.2) but just above our threshold ($p = 0.006$), because TD3 without layer normalization varies a lot between seeds: 4 of its 10 runs end with a negative return, against 1 of 10 with layer normalization. Layer normalization does not significantly change the return of either algorithm (DDPG: $p = 0.19$, TD3: $p = 0.12$). No condition reaches the threshold of 200 on average.

Table 2: Final return and critic bias (Q - Monte Carlo return) per condition, mean ± std over 10 seeds. Final values: mean of the last three evaluations (280k-300k steps). Mean bias: average over the evaluations from 20k to 300k steps.

Algorithm	Critic	Final return	Final bias	Mean bias
DDPG	no LN	-80.2 ± 56.6	184.3 ± 40.3	220.1 ± 57.7
DDPG	with LN	-118.1 ± 66.3	435.7 ± 104.1	357.1 ± 53.4
TD3	no LN	60.6 ± 123.0	-14.5 ± 13.3	-11.8 ± 13.0
TD3	with LN	138.6 ± 83.4	-15.4 ± 8.2	-6.8 ± 7.5

Bias and performance (Q2). Across the 40 runs, a higher final bias goes with a lower final return (Spearman $\rho = -0.82$, $p < 0.001$, Figure 2). But the runs form two groups, one per algorithm: TD3 runs have a bias between -37 and 1 and a return between -110 and 233, DDPG runs a bias between 133 and 597 and a return between -187 and 33. Inside each algorithm the correlation is weaker but still negative ($\rho = -0.50$, $p = 0.026$ for DDPG; $\rho = -0.59$, $p = 0.006$ for TD3).

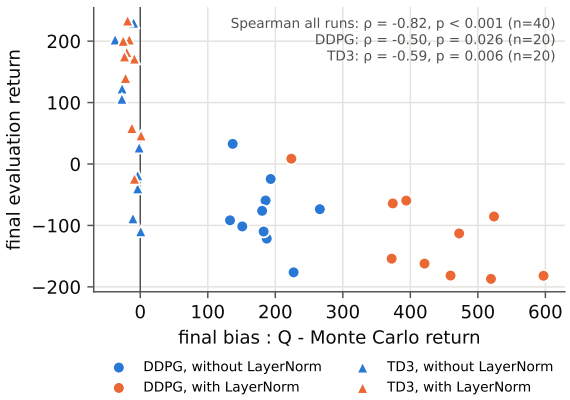


Figure 2: Final bias against final return, one point per run (40 runs). Color: with or without layer normalization; marker: algorithm. Final values are the mean of the last three evaluations (280k-300k steps).

4 Discussion

Layer normalization and overestimation. Contrary to our hypothesis H1a, layer normalization did not reduce the overestimation of DDPG: it more than doubled its final bias (184 to 436). A possible explanation is the amount of training per collected sample: Nauman et al. [5] observed the benefit of layer normalization with many gradient updates per environment step, whereas we perform a single one, so the setting in which layer normalization helps may not be reached in our experiments. For TD3, layer normalization had no measurable effect on the bias, which stayed slightly negative in both conditions (H1b). Since clipped double Q-learning already removes the overestimation, layer normalization has little left to correct.

Overestimation and performance. Across all runs, a larger bias goes with a lower return ($\rho = -0.82$), consistent with H2. However, the runs form one group per algorithm, and the two algorithms differ in more than their bias: TD3 uses two critics, as well as target policy smoothing and delayed updates. The correlation across all runs therefore cannot be attributed to the bias alone. Within each algorithm the correlation is weaker ($\rho = -0.50$ for DDPG, -0.59 for TD3), and it is hard to interpret: within TD3 the bias varies by only a few tens between runs while the return varies by hundreds, and a correlation does not show the direction of the effect, since a policy that lands and a policy that crashes visit different states, which changes the measured bias. Our results are thus compatible with H2 but do not show that overestimation causes the lower performance.

Limitations. Our bias estimate uses a single rollout per state, which makes each measurement noisy. Runs are limited to $3 \cdot 10^5$ steps and no condition reaches the solving threshold of 200, so the conclusions may differ for longer training. Finally, all hyperparameters are fixed, with one gradient update per environment step; varying the update-to-data ratio would directly test whether layer normalization reduces overestimation in the regime where it was previously reported to help.

Conclusion. In our setting, TD3 removes the overestimation of DDPG with or without layer normalization, while layer normalization increases the overestimation of DDPG instead of reducing it. A larger bias goes with a lower performance, but this relation is mostly a difference between the two algorithms.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[3] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, 2018.

[4] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2016.

[5] Michal Nauman, Michał Bortkiewicz, Piotr Miłoś, Tomasz Trzciński, Mateusz Ostaszewski, and Marek Cygan. Overestimation, overfitting, and plasticity in actor-critic: the bitter lesson of reinforcement learning. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[6] Mark Towers et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.